

BCSE102L Structured and Object Oriented Programming

Dr. Pramod Kumar Maurya

Assistant Professor (Senior)

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore, Tamil Nadu, India

Cabin: SJT405B

Email: pramodkumar.maurya@vit.ac.in

+91 9643322724

BCSE102L - Course Objectives

L	T	P	C
2	0	0	2

1. To impart the basic constructs in structured programming and object-oriented programming paradigms.
2. To inculcate the insights and benefits in accessing memory locations by implementing real world problems.
3. To help solving real world problems through appropriate programming paradigms.

BCSE102L - Course Outcomes

At the end of the course, students should be able to:

1. Understand different programming language constructs and decision-making statements; manipulate data as a group.
2. Recognize the application of modular programming approach; create user defined data types and idealize the role of pointers.
3. Comprehend various elements of object-oriented programming paradigm; propose solutions through inheritance and polymorphism; identify the appropriate data structure for the given problem and devise solution using generic programming techniques.

Course Syllabus

Module:1	C Programming Fundamentals	2 hours
Variables - Reserved words – Data Types – Operators – Operator Precedence - Expressions - Type Conversions - I/O statements - Branching and Looping: if, if-else, nested if, if-else ladder, switch statement, goto statement - Loops: for, while and do...while – break and continue statements.		
Module:2	Arrays and Functions	4 hours
Arrays: One Dimensional array - Two-Dimensional Array – Strings and its operations. User Defined Functions: Declaration – Definition – call by value and call by reference - Types of Functions - Recursive functions - Storage Classes - Scope, Visibility and Lifetime of Variables.		
Module:3	Pointers	4 hours
Declaration and Access of Pointer Variables, Pointer arithmetic – Dynamic memory allocation – Pointers and arrays - Pointers and functions.		
Module:4	Structure and Union	2 hours
Declaration, Initialization, Access of Structure Variables - Arrays of Structure - Arrays within Structure - Structure within Structures - Structures and Functions – Pointers to Structure -		

Course Syllabus

Module:5	Overview of Object-Oriented Programming	5 hours
Features of OOP - Classes and Objects - “this” pointer - Constructors and Destructors - Static Data Members, Static Member Functions and Objects - Inline Functions – Call by reference - Functions with default Arguments - Functions with Objects as Arguments - Friend Functions and Friend Classes.		
Module:6	Inheritance	5 hours
Inheritance - Types of Inheritance: Single inheritance, Multiple Inheritance, Multi-level Inheritance, Hierarchical Inheritance - Multipath Inheritance - Inheritance and constructors.		
Module:7	Polymorphism	4 hours
Function Overloading - Operator Overloading – Dynamic Polymorphism - Virtual Functions - Pure virtual Functions - Abstract Classes.		
Module:8	Generic Programming	4 hours
Function templates and class templates, Standard Template Library.		

Text Books

Text Book(s)	
1.	Herbert Schildt, C: The Complete Reference, 4 th Edition, McGraw Hill Education, 2017
2.	Herbert Schildt, C++: The Complete Reference, 4 th Edition, McGraw Hill Education, 2017.
Reference Books	
1.	Yashavant Kanetkar, Let Us C: 17 th Edition, BPB Publicaitons, 2020.
2.	Stanley Lippman and Josee Lajoie, C++ Primer, 5 th Edition, Addison-Wesley publishers, 2012.

BCSE102L Assessment Details

1. Digital Assignment 1 – 10 Marks
2. Technical Quiz 1 – 10 Marks
3. Technical Quiz 2 – 10 Marks
4. CAT 1 – 15 Marks
5. CAT 2 – 15 Marks
6. FAT – 40 Marks

BCSE102P List of Experiments

L	T	P	C
0	0	4	2

Indicative Experiments	
1.	Programs using basic control structures, branching and looping
2.	Experiment the use of 1-D, 2-D arrays and strings and Functions
3.	Demonstrate the application of pointers
4.	Experiment structures and unions
5.	Programs on basic Object-Oriented Programming constructs.
6.	Demonstrate various categories of inheritance
7.	Program to apply kinds of polymorphism.
8.	Develop generic templates and Standard Template Libraries.

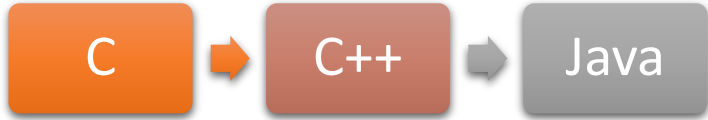
BCSE102P Assessment Details

1. Lab Assignment 1 – 10 Marks (From C Language)
2. Lab Assignment 2 – 10 Marks (From C Language)
3. Lab Assignment 3 – 10 Marks (From C++ Language)
4. Lab Assignment 4 – 10 Marks (From C++ Language)
5. Mid-Term Test – 20 Marks (Both C & C++ Language)
6. FAT – 40 Marks (Both C & C++ Language)

History of C

- It is a most popular language.
- Structured and machine independent language.
- C was evolved from **ALGOL -> CPL -> BCPL -> B -> C**
- Unix is one of the most popular network operating system and it was coded entirely in C

Who? When? Where? Why?

- Who? **Dennis Ritchie**
- Where? AT & T's Bell Laboratories of USA
- When? 1972.
- Why? C is reliable, friendly
- What next?
 - C++, Java => OOP

Why C is Middle Level Language ?

- C Programming supports Inline Assembly Language Programs
- Using inline assembly language feature in C, we can directly access system registers.
- C Programming is used to access memory directly using pointer.
- C Programming also Supports high Level Language Features.
- It is more User friendly as compare to Previous languages, so C programming is Middle Level Language.

Why C is Middle Level Language ?

- C is called middle-level language because it is actually bind the gap between a machine level language and high-level languages.
- User can use c language to do **System Programming** (for writing operating system) as well as **Application Programming** (for generate menu driven customer billing system).

That's why it is called middle level language.

- **High level** - Ada , Modula-2 , Pascal, COBOL, FORTRAN, BASIC
- **Middle level** - Java, C++, C, FORTH, Macro-assemble
- **Low level** - Assembler

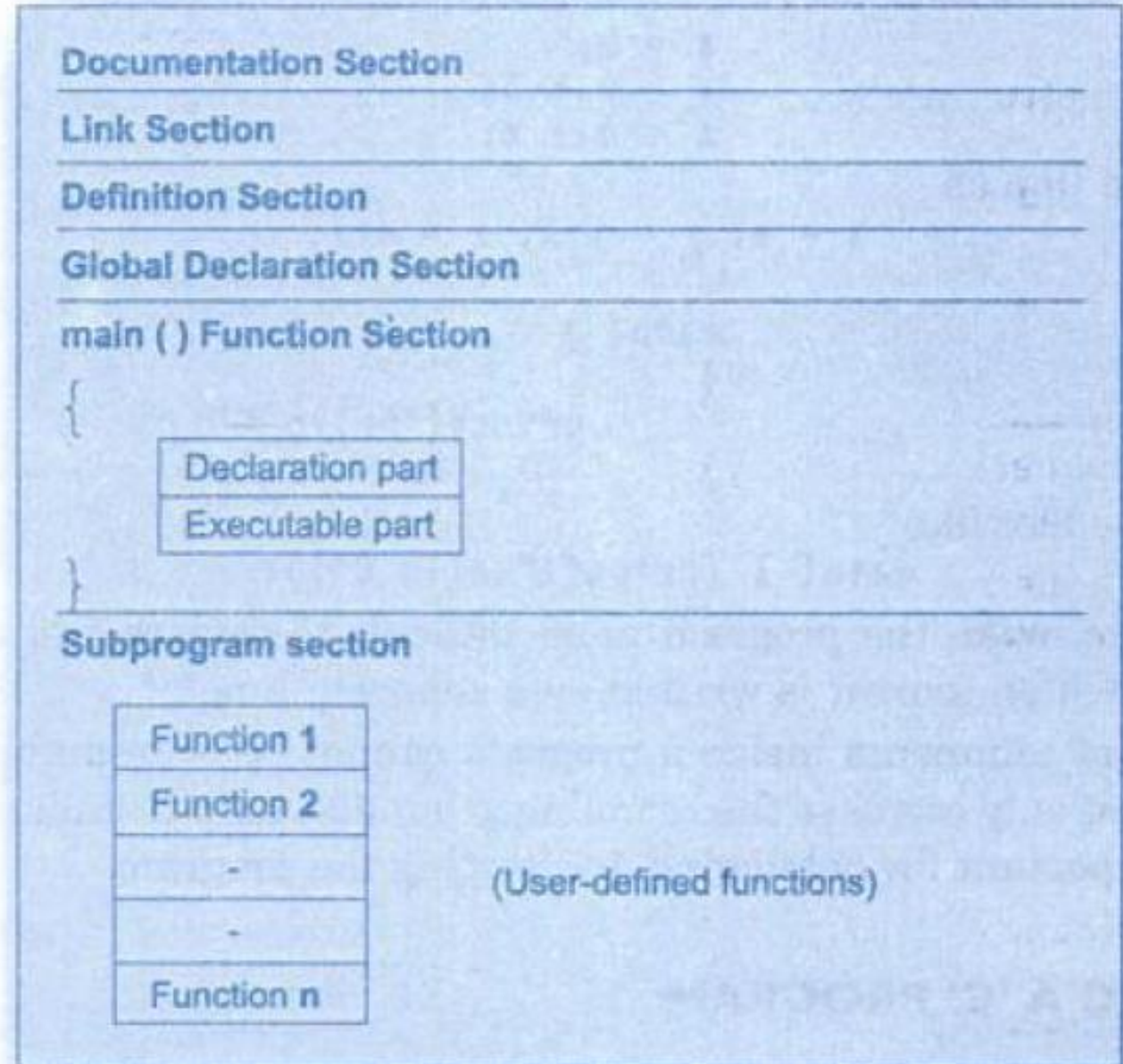
Features of C

1. General-purpose Structured language
2. Free format and case-sensitive language
3. System Programming language
4. Fast running and efficient language
5. Machine independent and hence it is portable
6. Supports limited data-types
7. Commands can be inserted anywhere in the program
8. Memory address are directly accessed by using pointers
9. Supports modularity
10. Permits recursion
11. More built-in functions

All the world is in “C”

- OS – Windows / UNIX / LINUX
- Device Drivers
- Mobile Devices
- Microprocessor in washing machine, digital camera, microwave oven
- 3D Computer Games - **still written in C**
- C is the best when it comes to performance

Structure of C Program



An Example C Program

```
main( )  
{  
printf("Welcome to BCSE102L");  
}
```

```
// Function name  
// Start of the program  
// Program statements  
// End of program
```

Output

Welcome to BCSE102L

Another Example

```
main( )
```

```
{
```

```
int number;
```

```
int amount;
```

```
number=100;
```

```
amount = 50 + 40;
```

```
printf(“%d”, number);
```

```
printf(“%d”, amount);
```

```
}
```

```
// Start of the program
```

```
// Declaration
```

```
// assignment statement
```

```
// printing the output
```

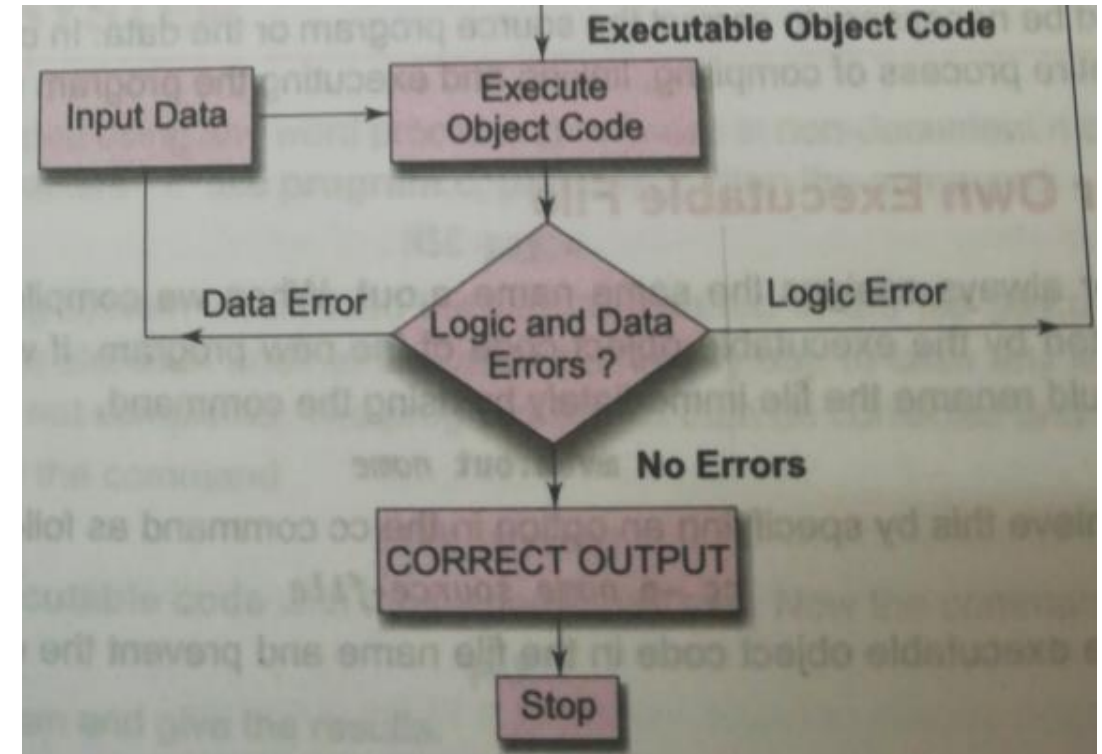
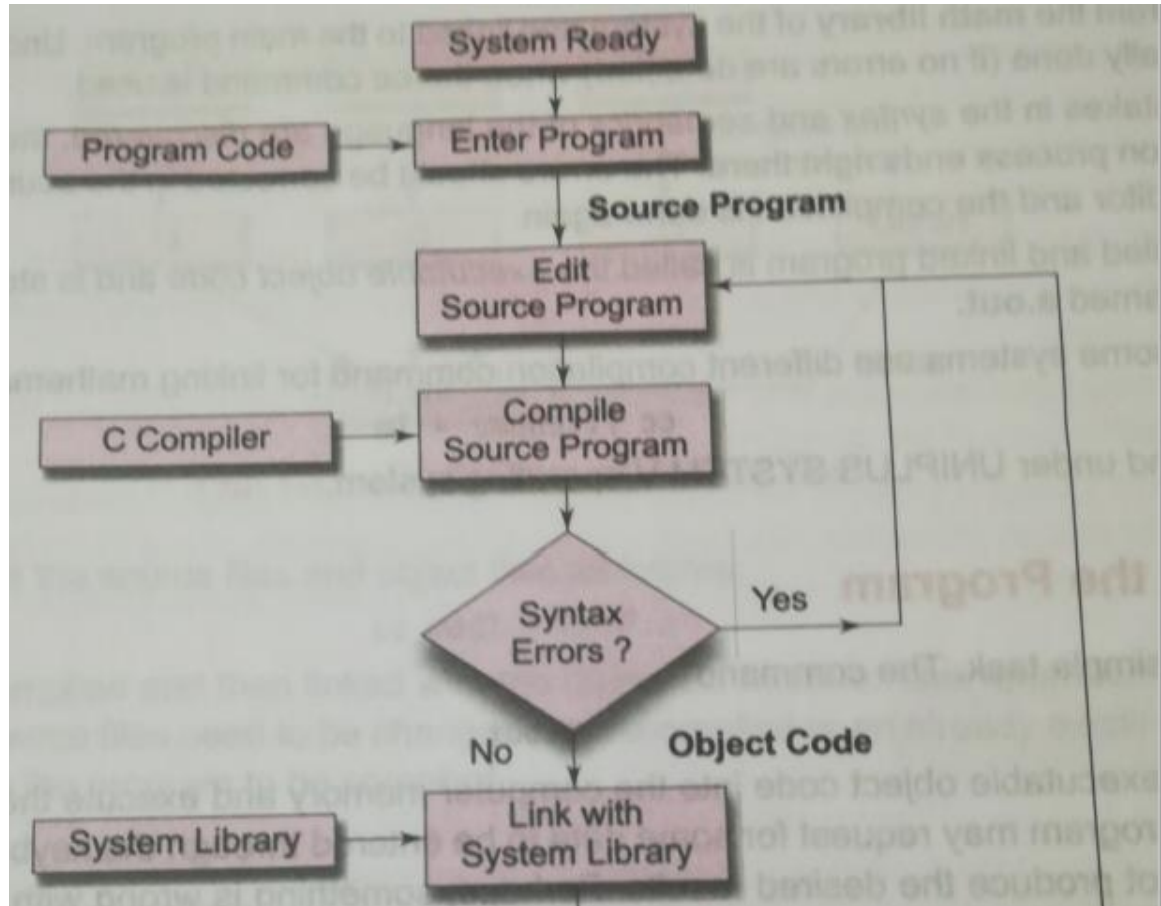
```
// End of program
```

Output

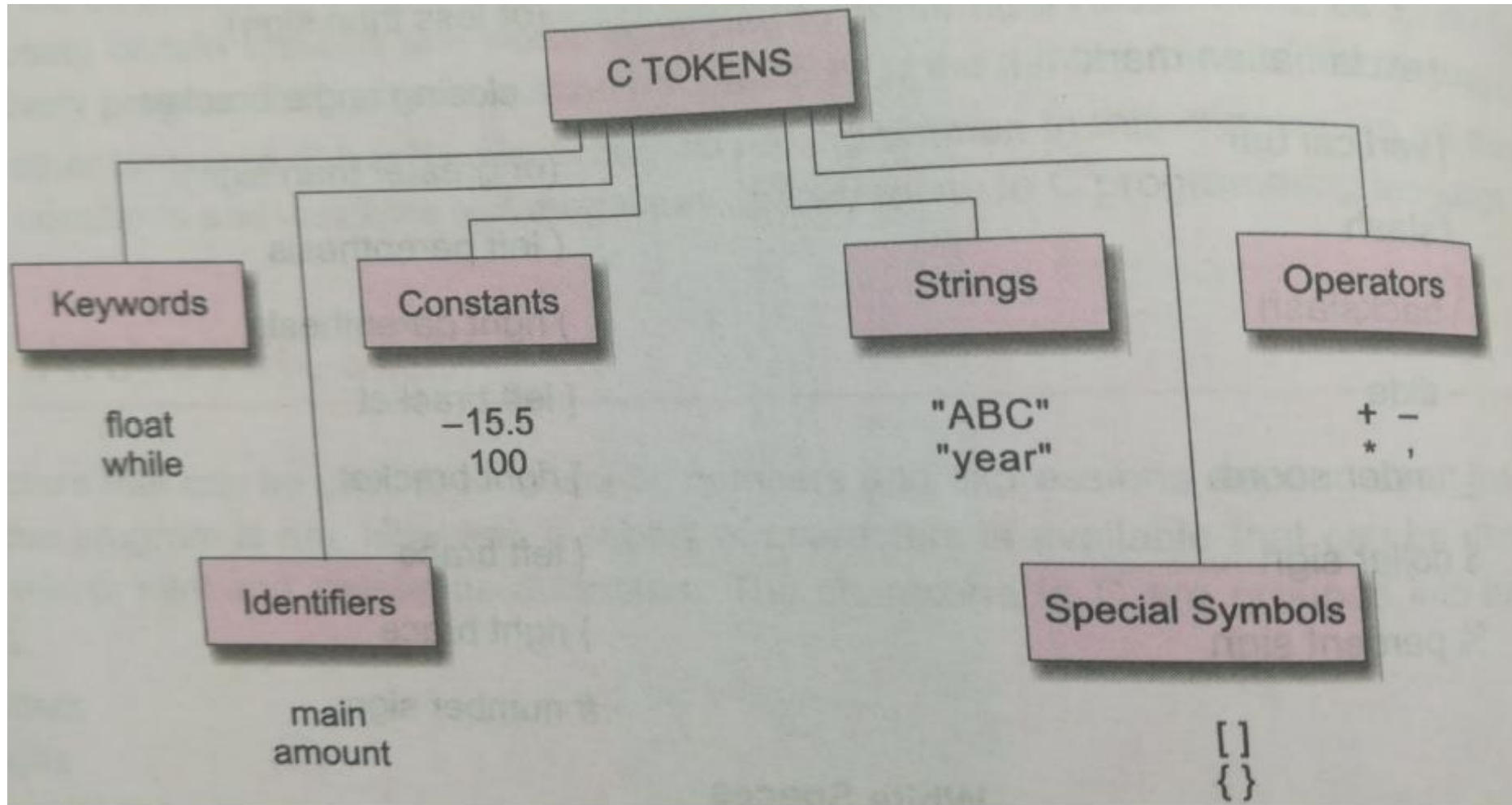
100

90

Execution Process



C Tokens



Keywords of ANSI C

<i>auto</i>	<i>double</i>	<i>int</i>	<i>struct</i>
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Note: Keywords should not be used as Variable Names

Identifiers

Identifiers refer to names of variables, functions and arrays

These are user-defined names and consist of sequence of letters and digits

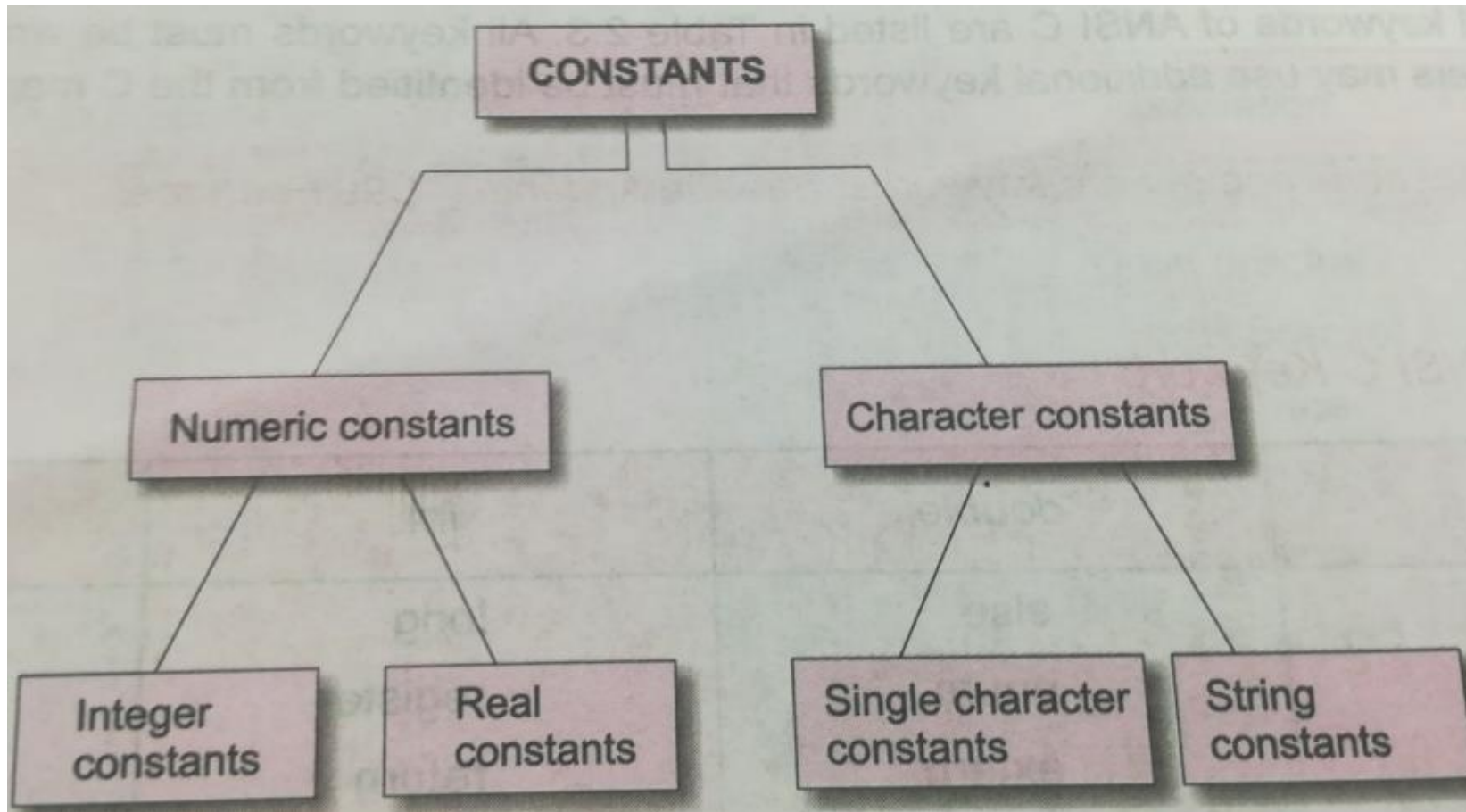
Example: int `sample`;

Rules

1. First character should be an alphabet
2. Must contain only letters, digits or underscore
3. Only first 31 characters are significant
4. Cannot use a keyword
5. Must not contain White space

Constants

Constants hold fixed values that do not change during the execution of the program.



1. Integer Constant

An integer constant refers to a sequence of digits.

Eg: 426, -8900

Rules

- It must have at least one digit.
- It must not have a decimal point.
- It can be either positive or negative.
- If no sign precedes an integer constant it is assumed to be positive.
- No commas or blanks are allowed within an integer constant.
- The allowable range for integer constants is -32768 to 32767.

Integer Constant

Three types:

1. Decimal integer – consists of set of digits, 0 through 9 (Eg. 129, -921, 0)
2. Octal integers - consists of set of digits, 0 through 7 with a leading 0. (Eg. 0123, 051)
3. Hexadecimal integers - consists of set of digits, 0 through 9 and A through F. Digits are preceded by 0x or 0X (Eg. 0x123, 0Xbcd, 0X9F)

2. Floating Point or Real Constant

1. Integers are inadequate to represent quantities that can vary continuously such as distances, heights, temperatures, prices, etc.
2. These quantities are represented by numbers containing fractional parts like 15.234

Rules

- It must have at least one digit.
- It must have a decimal point.
- It could be either positive or negative.
- Default sign is positive.
- No commas or blanks are allowed within a real constant.

3. Character Constant

- It is a single alphabet, a single digit or a single special symbol enclosed within single inverted commas.
- The maximum length of a character constant can be 1 character.

Eg: 'A' '5' '%'

Character constant have integer values known as ASCII values.

Eg. The ASCII value of the letter a is 97

So,

`printf("%d", 'a')` -> would print number 97

`printf("%c", '97')` - > would print letter 'a'

4. String Constant

- It is a sequence of characters enclosed in double quotes.
- The characters may be letters, numbers, special characters and blank space.

Eg: “Computer” “Computer Science” “2022”

Variables in C

- A variable is a data name used to store a data value
- It may take different values at different times during the execution

Rules

1. Must begin with a letter
2. It may consists of letters, digits and underscore
3. ANSI recognises a length of 31 characters but most compilers recognize only 8 characters
4. Case sensitive
5. It should not be keyword
6. White space is not allowed

Variables in C

- A variable is a data name used to store a data value
- It may take different values at different times during the execution

Rules

1. Must begin with a letter
2. It may consists of letters, digits and underscore
3. ANSI recognises a length of 31 characters but most compilers recognize only 8 characters
4. Case sensitive
5. It should not be keyword
6. White space is not allowed

Variables in C

Examples

<i>Variable name</i>	<i>Valid ?</i>	<i>Remark</i>
First_tag	Valid	
char	Not valid	char is a keyword
Price\$	Not valid	Dollar sign is illegal
group one	Not valid	Blank space is not permitted
average_number	Valid	First eight characters are significant
int_type	Valid	Keyword may be part of a name

Declaration and initialization of Variables

Syntax: data-type variable_name;

`int student_marks;` → Declaring

`float s1, s2, s3;`

`int student_marks = 48;` → assigning a value

Data types in C

ANSI C supports three classes of datatypes.

1. Primary data types
2. Derived data types
3. User-defined data types

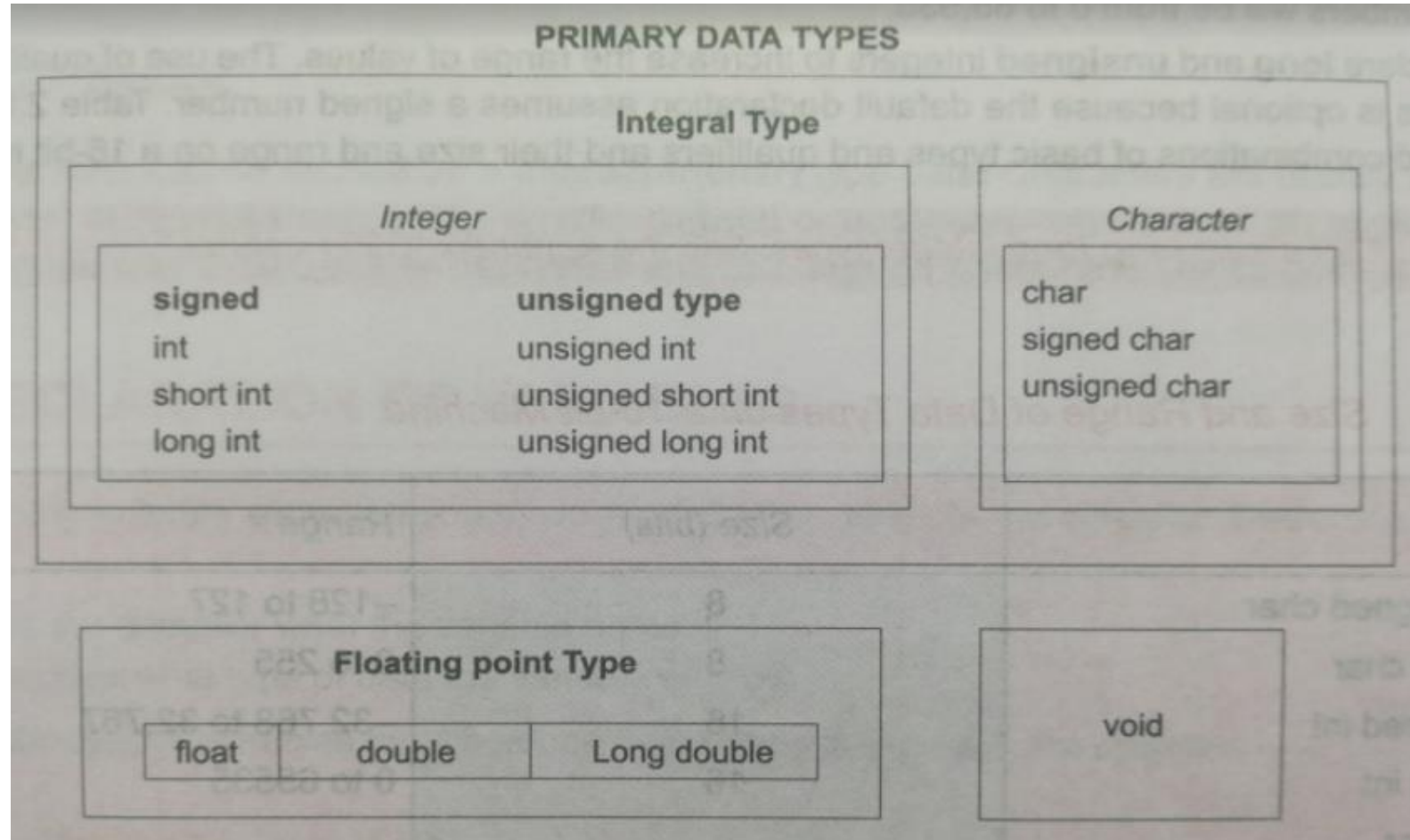
Primary data types

Supports five fundamental data types

Integer (int), Character (char), floating point (float), double-precision floating point (double) and void

Data types in C

Primary data types



Data types in C

Primary data types

Type	Size (bits)	Range
char or signed char	8	−128 to 127
unsigned char	8	0 to 255
int or signed int	16	−32,768 to 32,767
unsigned int	16	0 to 65535
short int or		
signed short int	8	−128 to 127
unsigned short int	8	0 to 255
long int or		
signed long int	32	−2,147,483,648 to 2,147,483,647
unsigned long int	32	0 to 4,294,967,295
float	32	$3.4E - 38$ to $3.4E + 38$
double	64	$1.7E - 308$ to $1.7E + 308$
long double	80	$3.4E - 4932$ to $1.1E + 4932$

Primary Data Types

Data Type	Range	Bytes	Format
signed char	-128 to + 127	1	%c
unsigned char	0 to 255	1	%c
short signed int	-32768 to +32767	2	%d
short unsigned int	0 to 65535	2	%u
signed int	-32768 to +32767	2	%d
unsigned int	0 to 65535	2	%u
long signed int	-2147483648 to +2147483647	4	%ld
long unsigned int	0 to 4294967295	4	%lu
float	-3.4e38 to +3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to +1.7e4932	10	%Lf
Note: The sizes and ranges of int, short and long are compiler dependent. Sizes in this figure are for 16-bit compiler.			

Defining Symbolic Constants

A constant value can be assigned to these names at the beginning of the program

Syntax: `#define Symbolic-name value-of-constant`

Example: `#define STRENGTH 100`
 `#define PASSMARK 50`